Generative Models

Machine Learning course notes Sources: listed under References

What this file is really about:

$$\text{fit a distribution} \rightarrow \text{compress} \rightarrow \text{make the code sampleable} \rightarrow \text{learn a judge} \rightarrow \text{undo noise}$$

How to use this file. Part A builds each family on small examples. Part B shows the core code. Part C is practice with answers.

Step-up path

You need. Neural networks, normals and KL divergence.

1. Fit and sample a distribution.
2. Compress with an autoencoder.
3. Make codes sampleable with a VAE.
4. Train against a judge with a GAN.
5. Reverse noise with diffusion.

You can stop when you can compare the families and derive the VAE objective.

Contents

Part A Maths From the Ground Up

A1 What it means to generate

A classifier learns $p(y \mid x)$. A generative model learns about $p(x)$, so it can draw new examples. Fitting a Gaussian or a mixture to data and sampling from it is already generative. Images need something far more flexible.

A2 Autoencoders

An **autoencoder** squeezes x through a narrow code $z = f(x)$ and rebuilds it as $\hat{x} = g(z)$, minimising the reconstruction error. A linear autoencoder with squared loss learns the same subspace as PCA.

> **Careful.** A plain autoencoder is a poor generator: its codes can be scattered, so a random point in code space may decode to nonsense.

A3 Kullback-Leibler divergence

$$\mathrm{KL}(q \| p) = \mathbb{E}_q\left[\log \frac{q(z)}{p(z)}\right] \geq 0,$$

with equality only when $q = p$. For a normal against a standard normal,

$$\mathrm{KL}\big(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0, 1)\big) = \tfrac{1}{2}\big(\mu^2 + \sigma^2 - \ln \sigma^2 - 1\big).$$

A4 The variational autoencoder

Make the encoder output a distribution $q(z \mid x)$, and ask that it stay close to a simple prior. The objective is the **evidence lower bound**:

$$\log p(x) \geq \mathbb{E}_{q(z|x)}[\log p(x \mid z)] - \mathrm{KL}\big(q(z \mid x) \,\|\, p(z)\big).$$

The first term rewards good reconstruction, and the second keeps the codes organised so that sampling from the prior works.

The reparameterisation trick. We cannot backpropagate through a random draw, so we write $z = \mu + \sigma\varepsilon$ with $\varepsilon \sim \mathcal{N}(0, 1)$. The randomness moves into an input, and gradients flow through μ and σ.

A5 Generative adversarial networks

A **GAN** skips the likelihood. A generator G turns noise into samples, and a discriminator D tries to tell real from fake:

$$\min_G \max_D \; \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))].$$

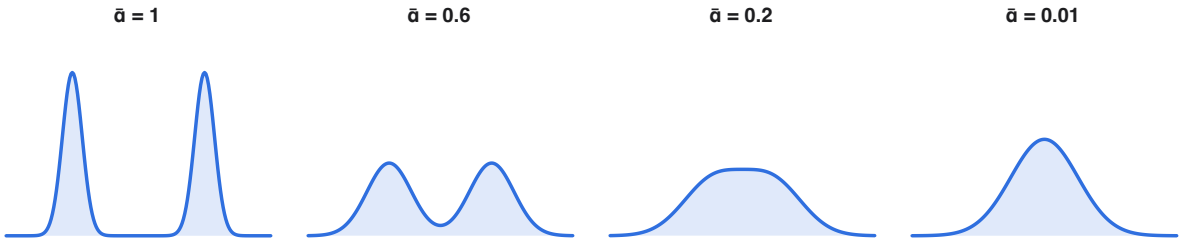
For a fixed generator the best discriminator is $D^*(x) = p_{\text{data}}(x)/(p_{\text{data}}(x) + p_G(x))$. Samples are sharp, but training is unstable, and **mode collapse** occurs when the generator covers only part of the data.

A6 Diffusion: destroying data

Add a little noise, many times. With $\bar{\alpha}_t = \prod_{s \le t}(1 - \beta_s)$, any step can be reached directly:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon.$$

As $\bar{\alpha}_t \to 0$, x_t becomes pure noise.



Two peaks blur into a single normal as the forward process runs.

A7 Diffusion: learning to undo

Train a network $\varepsilon_\theta(x_t, t)$ to predict the noise that was added, with the simple loss $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$. To generate, start from pure noise and remove predicted noise step by step. This loss follows from an evidence lower bound on the data, as with the VAE.

The score $\nabla_x \log p_t(x)$ points towards higher density, and the predicted noise is proportional to its negative, $\varepsilon_\theta \approx -\sqrt{1-\bar{\alpha}_t}\,\nabla_x \log p_t(x)$.

A8 Comparing the families

Family	Sampling	Sharpness	Training
VAE	fast	often blurry	stable
GAN	fast	sharp	unstable
Diffusion	slower	sharp	stable

A9 The evidence lower bound, derived

For any distribution $q(z \mid x)$,

$$\log p(x) = \log \int p(x,z)\,dz = \log \mathbb{E}_q\left[\frac{p(x,z)}{q(z \mid x)}\right] \geq \mathbb{E}_q\left[\log \frac{p(x,z)}{q(z \mid x)}\right]$$

by Jensen's inequality. Splitting $p(x,z) = p(x \mid z)p(z)$ gives

$$\mathrm{ELBO} = \mathbb{E}_q[\log p(x \mid z)] - \mathrm{KL}(q(z \mid x) \,\|\, p(z)).$$

The gap is exactly $\mathrm{KL}(q(z \mid x) \| p(z \mid x))$, so a better encoder tightens the bound.

A10 The closed-form KL, derived

For $q = \mathcal{N}(\mu, \sigma^2)$ and $p = \mathcal{N}(0,1)$:

$$\mathrm{KL} = \mathbb{E}_q\left[-\log \sigma - \frac{(z-\mu)^2}{2\sigma^2} + \frac{z^2}{2}\right].$$

Using $\mathbb{E}_q[(z-\mu)^2] = \sigma^2$ and $\mathbb{E}_q[z^2] = \mu^2 + \sigma^2$,

$$\mathrm{KL} = \tfrac{1}{2}\left(\mu^2 + \sigma^2 - \log \sigma^2 - 1\right).$$

A11 The GAN optimum, derived

For a fixed generator, the discriminator maximises, at each x,

$$p_{\text{data}} \log D + p_g \log(1 - D).$$

Setting the derivative to zero gives $D^* = p_{\text{data}}/(p_{\text{data}} + p_g)$. Substituting back, the generator's objective becomes

$$2\,\mathrm{JS}(p_{\text{data}} \| p_g) - \log 4.$$

So the generator is minimising a Jensen-Shannon divergence, and the ideal outcome is $p_g = p_{\text{data}}$ with $D = \frac{1}{2}$.

A12 Why GANs are hard

Mode collapse means the generator covers only part of the data. **Vanishing gradients** occur when the discriminator wins outright, because JS is flat for distributions that do not overlap. **Wasserstein GANs** use a distance that keeps useful gradients even then, and non-saturating losses and spectral normalisation help stability.

Part B In Code

B1 VAE loss

```
z = mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)
kl = -0.5 * (1 + logvar - mu**2 - logvar.exp()).sum(1).mean()
loss = F.mse_loss(decoder(z), x, reduction='sum') / len(x) + kl
```

B2 Diffusion training step

```
t = torch.randint(0, T, (len(x),))
eps = torch.randn_like(x)
xt = alpha_bar[t].sqrt() * x + (1 - alpha_bar[t]).sqrt() * eps
loss = F.mse_loss(model(xt, t), eps)
```

B3 A complete VAE

```python
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)

def data(n):
    t = torch.rand(n) * 2 * math.pi
    return torch.stack([2 * t.cos(), 2 * t.sin()], 1) + 0.1 * torch.randn(n, 2)


class VAE(nn.Module):
    def __init__(self, z=2, h=64):
        super().__init__()
        self.enc = nn.Sequential(nn.Linear(2, h), nn.ReLU(), nn.Linear(h, h), nn.ReLU(), nn
        self.dec = nn.Sequential(nn.Linear(z, h), nn.ReLU(), nn.Linear(h, h), nn.ReLU(), nn

    def forward(self, x):
        mu, log_var = self.enc(x).chunk(2, 1)
        z = mu + (0.5 * log_var).exp() * torch.randn_like(mu)     # reparameterisation
        return self.dec(z), mu, log_var


vae = VAE()
opt = torch.optim.Adam(vae.parameters(), lr=2e-3)
for step in range(2500):
    x = data(256)
    rec, mu, log_var = vae(x)
    recon = ((rec - x) ** 2).sum(1).mean()
    kl = 0.5 * (mu ** 2 + log_var.exp() - log_var - 1).sum(1).mean()
    loss = recon + 0.1 * kl
    opt.zero_grad(); loss.backward(); opt.step()
```

```
with torch.no_grad():
    r = vae.dec(torch.randn(1000, 2)).norm(dim=1)
print('mean radius', round(r.mean().item(), 2), '(target 2.0)')
assert 1.5 < r.mean() < 2.5
```

B4 A complete GAN

```
import torch
from torch import nn

torch.manual_seed(0)
G = nn.Sequential(nn.Linear(1, 32), nn.ReLU(), nn.Linear(32, 1))
D = nn.Sequential(nn.Linear(1, 32), nn.ReLU(), nn.Linear(32, 1))
og = torch.optim.Adam(G.parameters(), lr=1e-3, betas=(0.5, 0.999))
od = torch.optim.Adam(D.parameters(), lr=1e-3, betas=(0.5, 0.999))
bce = nn.functional.binary_cross_entropy_with_logits
real_data = lambda n: 3 + 0.5 * torch.randn(n, 1)

for step in range(3000):
    real, fake = real_data(128), G(torch.randn(128, 1))
    loss_d = bce(D(real), torch.ones(128, 1)) + bce(D(fake.detach()), torch.zeros(128, 1))
    od.zero_grad(); loss_d.backward(); od.step()
    loss_g = bce(D(G(torch.randn(128, 1))), torch.ones(128, 1))    # non-saturating
    og.zero_grad(); loss_g.backward(); og.step()

s = G(torch.randn(2000, 1)).detach()
print('generated mean', round(s.mean().item(), 2), 'std', round(s.std().item(), 2), '(targe
assert 2.5 < s.mean() < 3.5
```

Part C Practice

P1. Find $\mathrm{KL}(\mathcal{N}(1,1) \,\|\, \mathcal{N}(0,1))$.

$\frac{1}{2}(1+1-0-1) = 0.5$.

P2. Why does a VAE need a KL term?

Without it the codes need not match the prior, so samples from the prior would decode poorly.

P3. $\beta = 0.1$ for two steps. Find $\bar{\alpha}_2$.

$0.9^2 = 0.81$.

P4. $x_0 = 2$, $\bar{\alpha} = 0.81$, $\varepsilon = 1$. Find x_t.

$0.9 \times 2 + \sqrt{0.19} = 2.236$.

P5. At a point where $p_{\text{data}} = p_G$, what is the best discriminator's output?

0.5.

P6. What is mode collapse?

The generator produces only a small part of the variety in the data.

P7. Find the KL for $\mu = 1$, $\sigma = 1$.

$\frac{1}{2}(1+1-0-1) = 0.5$.

P8. At the GAN optimum, what does the discriminator output?

$\frac{1}{2}$ everywhere.